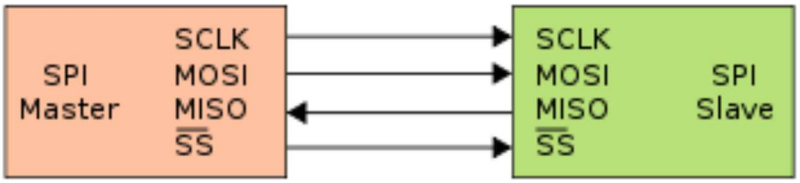
Lab 09 - SPI and LED Matrix

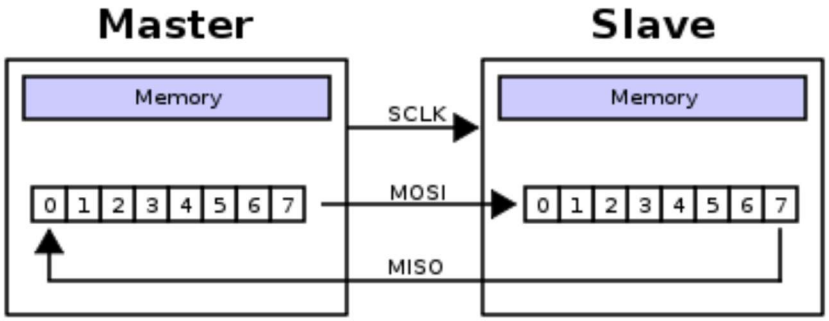
SPI – il protocollo

SPI (Serial Peripheral Interface) è un'interfaccia di comunicazione seriale **sincrona** (cioè accompagnata da un segnale di clock dedicato) pensata per comunicazioni a **corta distanza**, tipicamente tra un microcontrollore e uno o più periferici sulla **stessa scheda**.
Usa 4 segnali:

- **SCLK (Serial Clock)**: generato sempre dal master, scandisce quando i bit vengono spostati.
- **MOSI (Master Out, Slave In)**: dati dal master verso lo/gli slave.
- **MISO (Master In, Slave Out)**: dati dallo slave verso il master.
- **SS/CS (Slave Select / Chip Select)**: linea, tipicamente active-low, con cui il master "sveglia" lo slave con cui vuole parlare.



È **full-duplex**: master e slave si scambiano bit contemporaneamente sullo stesso fronte di clock. Il modo più semplice per capirlo è pensare a **due shift register collegati ad anello** (master e slave, tipicamente a 8 bit): a ogni colpo di clock, il master spinge fuori un bit su MOSI (che lo slave riceve) e *contemporaneamente* lo slave spinge fuori un bit su MISO (che il master riceve) — è la ***"virtual ring topology"***. I dati escono sempre **MSB first**.

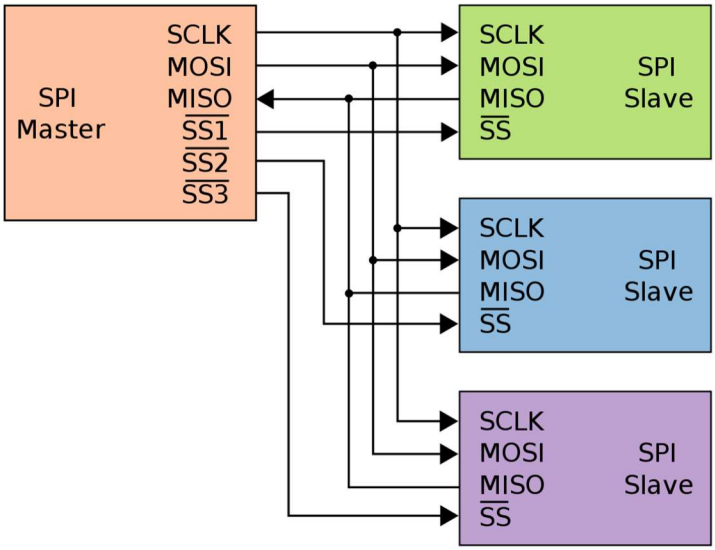


Nonostante si parli di "protocollo a 4 fili", i 4 fili non sono sempre tutti necessari (es. comunicazione solo in scrittura verso uno slave che non deve mai rispondere non ha bisogno di MISO), e molti componenti aggiungono pin ausiliari (reset, output enable...) oltre ai 4 canonici.

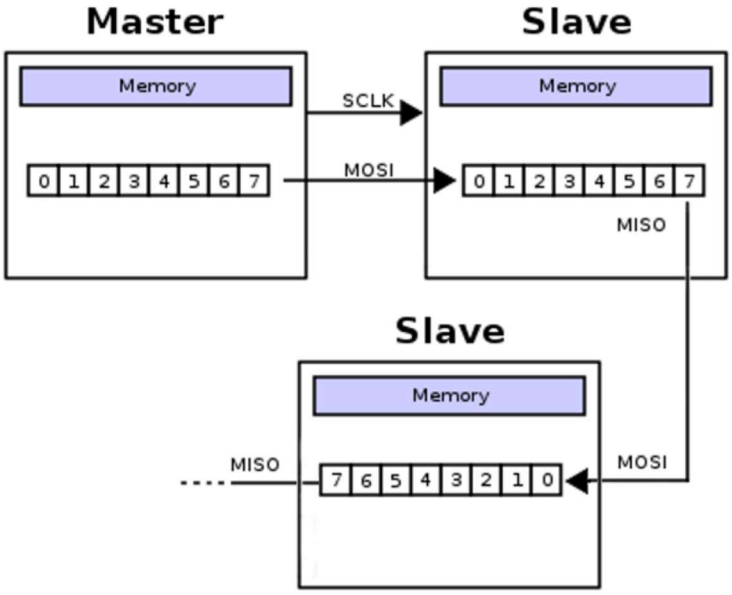
Configurazioni multi-slave

Con più slave sullo stesso bus ci sono due modi di gestirli:

- **Independent slave select**: **ogni slave ha la propria linea CS dedicata dal master**. Gli slave devono poter mettere MISO in alta impedenza (tri-state) quando non sono selezionati, altrimenti si scontrerebbero sulla stessa linea.



- **Daisy chain**: **un'unica linea CS condivisa da tutti**. Il MISO di uno slave è collegato al MOSI del successivo, quindi l'intera catena si comporta come un unico shift register lungo N×(word size). Per scrivere su un solo dispositivo della catena, bisogna comunque trasmettere una parola per *ogni* dispositivo della catena (mandando comandi NOP/idle a quelli che non si vogliono toccare). **Questa è esattamente la configurazione usata per la matrice LED del laboratorio.**



CPOL e CPHA

Questi due parametri definiscono il "dialetto" SPI usato:

- **CPOL (Clock Polarity)** — stato di riposo del clock: CPOL=0 → riposo basso, CPOL=1 → riposo alto.
- **CPHA (Clock Phase)** — su quale fronte viene campionato il dato: CPHA=0 → sul fronte "leading" (il primo, quello che esce dallo stato di riposo), CPHA=1 → sul fronte "trailing" (il secondo).

Le combinazioni danno i 4 **SPI mode**:

Mode	CPOL	CPHA	Clock a riposo	Campionamento
0	0	0	basso	fronte di salita
1	0	1	basso	fronte di discesa
2	1	0	alto	fronte di discesa
3	1	1	alto	fronte di salita

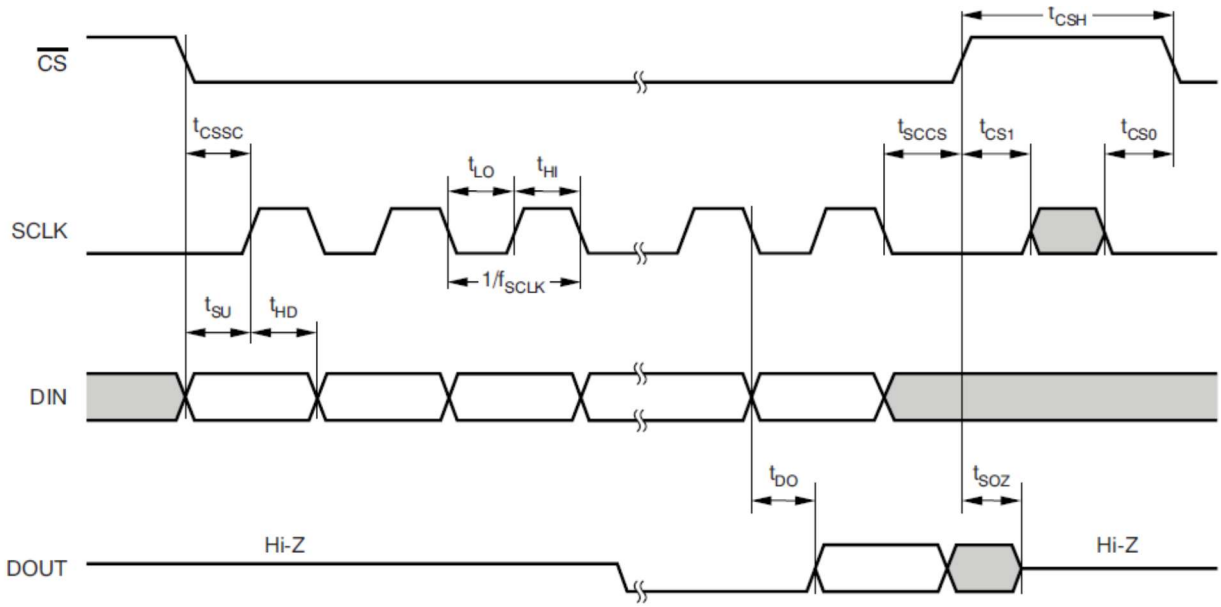


Figure 1. SPI Mode 0, 0

Nella configurazione CubeMX mostrata (CPOL=Low, CPHA="1 Edge" — attenzione: in HAL "1 Edge" corrisponde a CPHA=0, non 1) si sta usando il **Mode 0**: clock a riposo basso, campionamento sul fronte di salita. Non è un caso: è esattamente il comportamento richiesto dal 74HC595 (vedi sotto), che campiona SER sul fronte di salita di SHCP.

SPI sull'STM32F401

Il microcontrollore ha fino a 4 periferiche SPI: SPI1/SPI4 arrivano a 42 Mbit/s, SPI2/SPI3 a 21 Mbit/s (dipende dal clock APB a cui sono agganciate). Un **prescaler a 3 bit dà 8 possibili frequenze di clock in modalità master**; la dimensione della **parola è configurabile a 8 o 16 bit**; e tutte le SPI possono essere servite da **DMA**.
Le funzioni HAL principali:

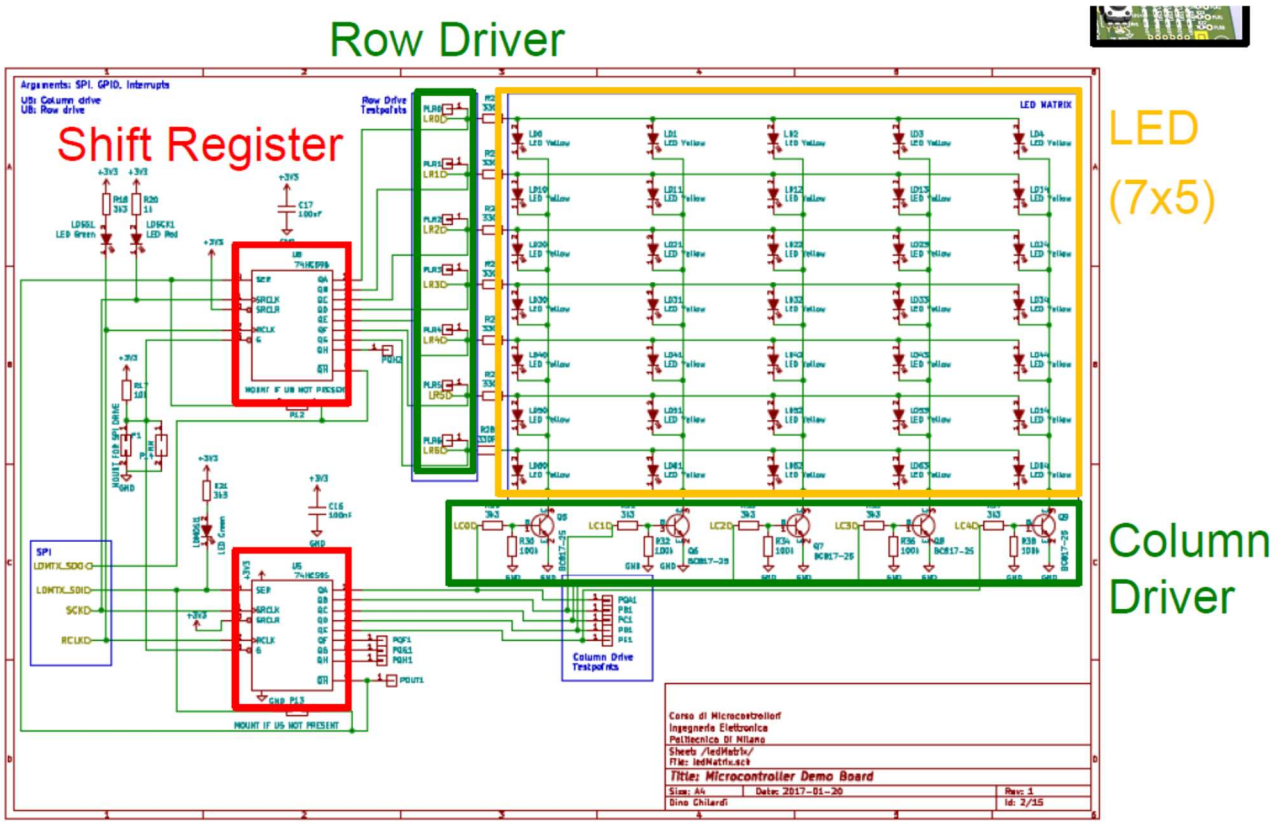
- `HAL_SPI_Transmit(&hspi, pData, Size, Timeout)` — bloccante.
- `HAL_SPI_Transmit_DMA(&hspi, pData, Size)` — non bloccante, il trasferimento avviene in background via DMA.

74HC595 — lo shift register esterno

Il **74HC595** è un integrato **8-bit serial-in, parallel-out shift register** con storage register interno. È il "traduttore" che trasforma il flusso seriale SPI in 8 linee parallele stabili. I suoi pin:

- **Serial Data Input (DS)**: ingresso dati seriale (collegato a MOSI).
- **Shift Clock SRCLK (SH_CP)**: clock dello shift register — ad ogni fronte di salita, il bit su SER entra nel registro e tutti gli altri bit scalano di una posizione (collegato a SCLK).
- **Store Clock RCLK (ST_CP)**: clock di *storage/latch* — sul fronte di salita, copia in blocco tutto il contenuto dello shift register nel registro di uscita (Q0-Q7), aggiornando i pin fisici. È **separato** da SRCLK apposta: permette di riempire lo shift register bit per bit senza che le uscite "sfarfallino" ad ogni singolo bit, e di aggiornare tutte le 8 uscite in un colpo solo quando i dati sono pronti.
- **Output Enable (OE)**: se basso, le uscite Q0-Q7 mostrano il contenuto del registro di storage; se alto, le uscite vanno in alta impedenza.
- **Output Pins (Q0-Q7)**: le 8 uscite parallele.
- **Serial Out (Q7S)**: riporta in uscita l'ultimo bit dello shift register (quello "più vecchio", spinto fuori dall'ottavo colpo di clock) — è il pin che permette di incatenare (daisy chain) un secondo 74HC595 collegando Q7S del primo a SER del secondo.
- **Reset Pin (MR)**: azzera lo shift register (non lo storage register).

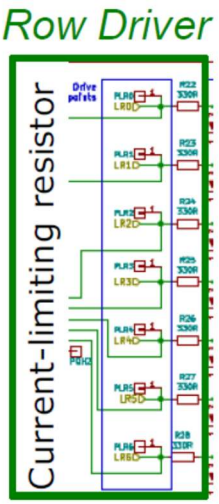
Nel laboratorio ne trovi **due in cascata**: uno per pilotare le righe della matrice, uno per le colonne, con SH_CP e ST_CP condivisi (perché sono la stessa linea SPI/latch per entrambi) ma con i dati seriali incatenati (Q7S del primo → DS del secondo).

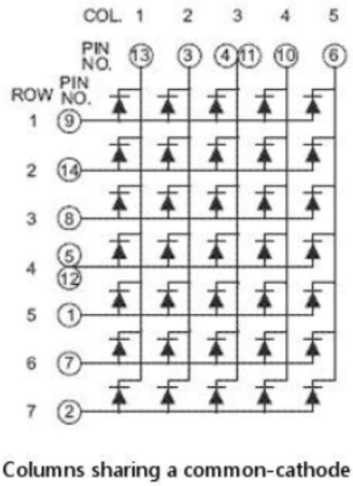


Matrice LED 7×5

La matrice non è pilotata "un LED alla volta" collegato singolarmente: i LED sono organizzati a griglia per usare pochi pin.

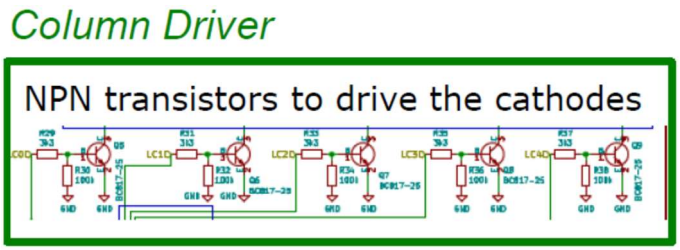
- **Row driver**: le uscite del primo 74HC595 passano attraverso resistenze di limitazione corrente e vanno direttamente agli anodi delle righe (nella configurazione della board, colonne a catodo comune per riga).

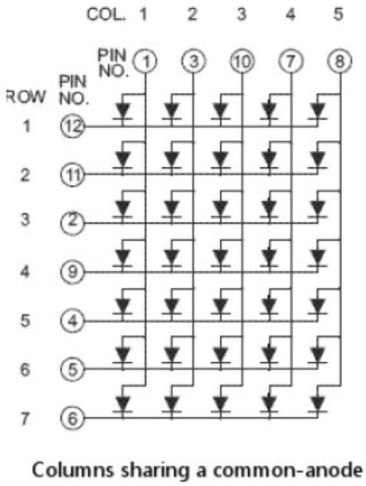




Columns sharing a common-cathode

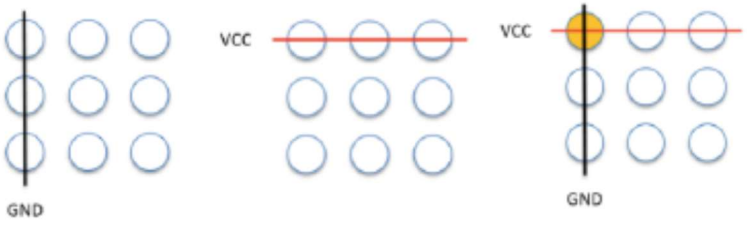
- **Column driver**: le uscite del secondo 74HC595 non pilotano i LED direttamente, ma le basi di **transistor NPN**, usati come interruttori controllati in tensione: quando un transistor viene acceso, collega a massa il catodo comune della sua colonna, e solo allora i LED di quella colonna (per cui l'anodo di riga è già alto) si accendono davvero.





Columns sharing a common-anode

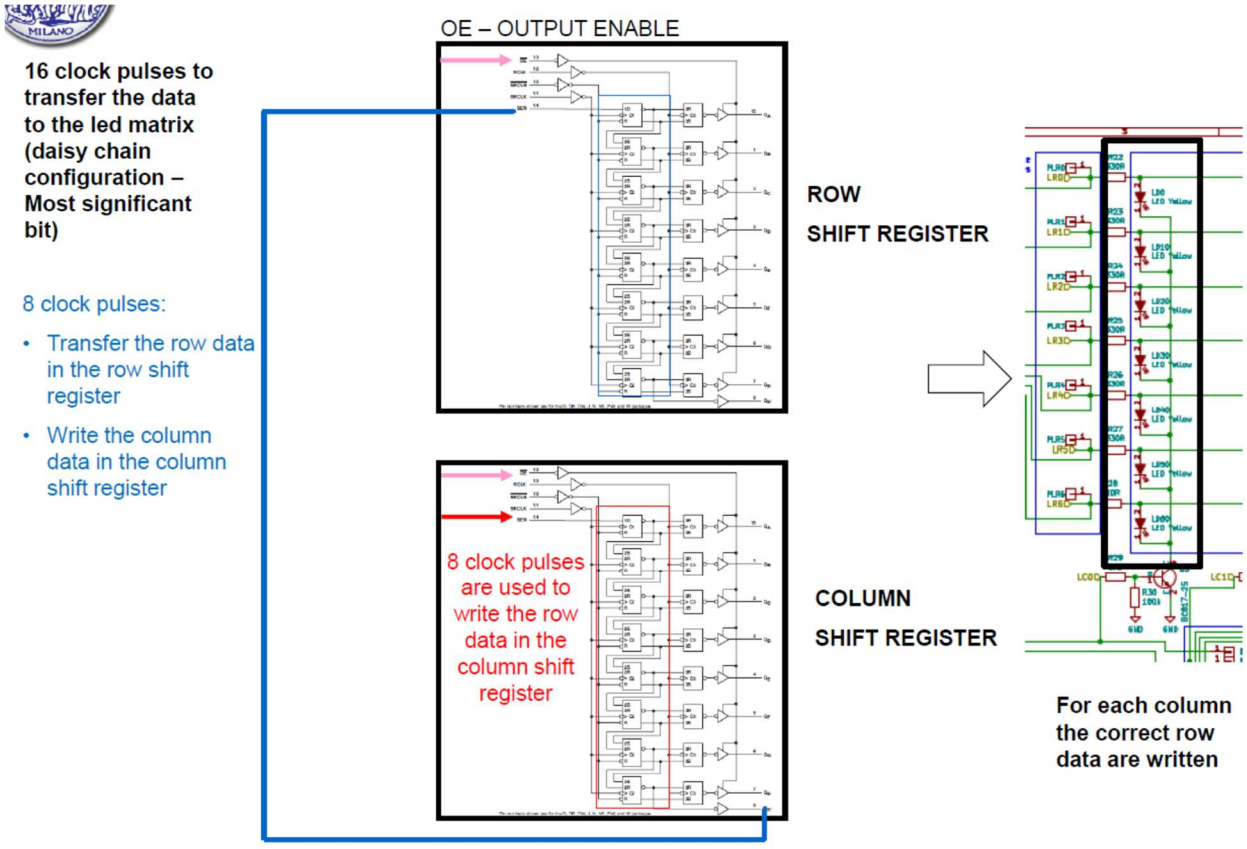
Il punto chiave è che **si può scrivere una sola colonna alla volta** (solo un transistor colonna acceso), quindi per mostrare un'immagine 2D completa bisogna fare **multiplexing**: si scandiscono velocemente tutte le colonne una dopo l'altra (accendendo, per ciascuna, solo le righe corrette), con un refresh rate abbastanza alto (>50 Hz, cioè <4 ms per colonna) da ingannare la persistenza della visione e far sembrare che l'intera matrice sia accesa insieme.



Il trasferimento dati (16 bit totali via daisy chain)

Per ogni colonna da aggiornare si mandano **16 impulsi di clock** in una singola trasmissione SPI (perché i due 74HC595 sono in daisy chain e si comportano come un unico shift register da 16 bit):

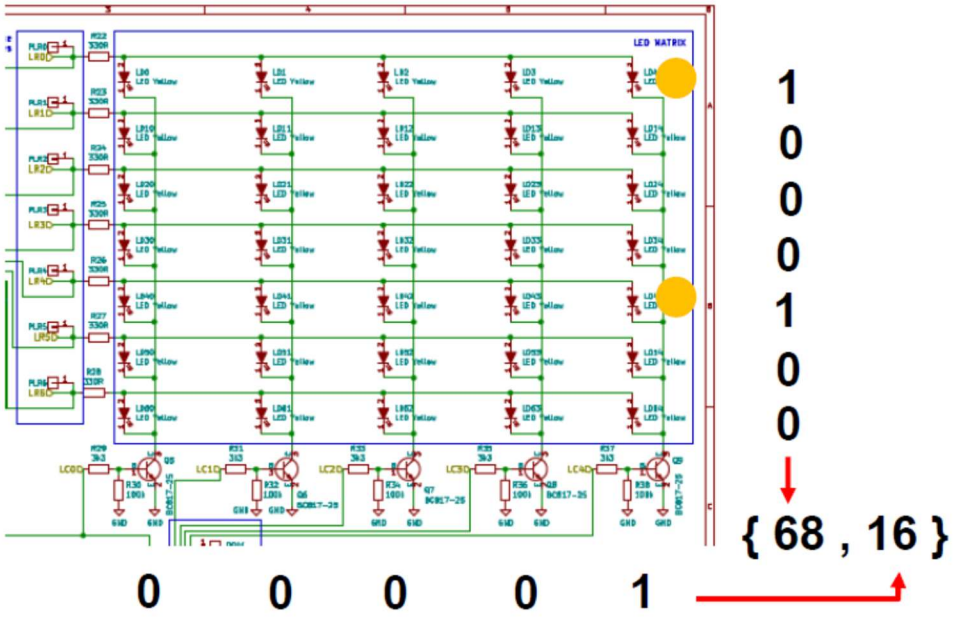
1. Si trasmettono 8 bit che rappresentano il pattern delle righe (quali LED di quella colonna devono accendersi).
2. Si continua a trasmettere altri 8 bit col valore della colonna da selezionare; nel frattempo, i primi 8 bit già inviati vengono spinti, tramite il collegamento a catena, nel secondo registro.
3. Fatti i 16 colpi di SRCLK, si dà **un solo impulso su RCLK** (comune a entrambi i 74HC595): questo fa scattare simultaneamente sia le righe che la colonna selezionata, evitando stati intermedi visibili (flic-kering).



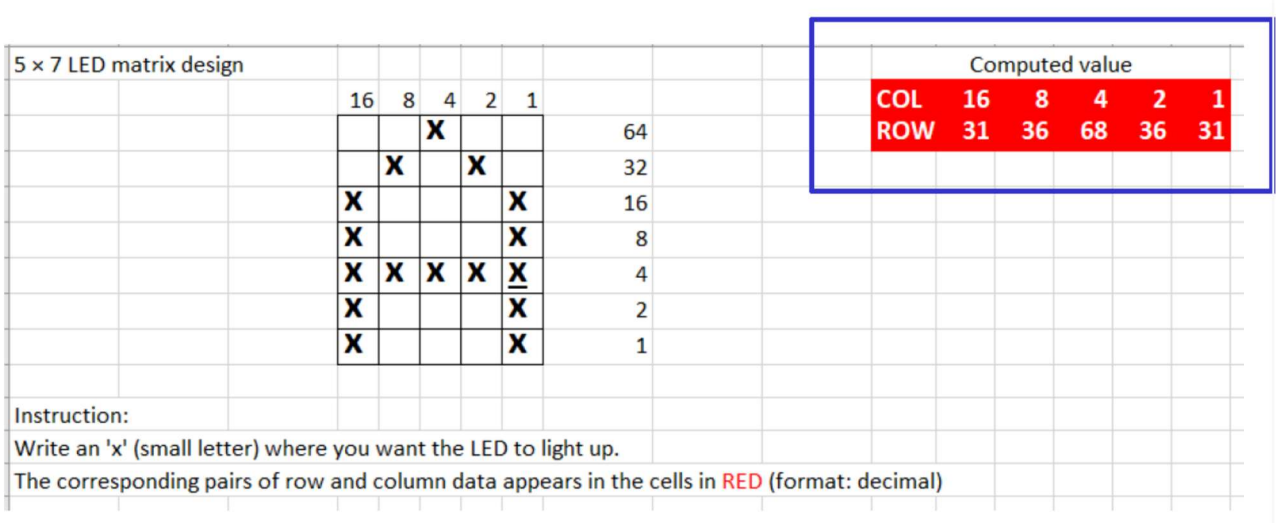
Codifica dei valori

Sia il **byte riga** che il **byte colonna** vengono interpretati come numeri binari con **pesi posizionali**:

- per le 5 colonne i pesi sono 16, 8, 4, 2, 1 (un solo bit a 1 per volta → un solo transistor colonna acceso).
- per le 7 righe i pesi sono 64, 32, 16, 8, 4, 2, 1 (qui più bit possono essere a 1 contemporaneamente, perché in una colonna possono accendersi più LED insieme).



Il file LED matrix computation.xlsx ti permette di disegnare un carattere su una griglia 5×7 segnando le celle con una "x" e ottenere direttamente le 5 coppie (riga, colonna) da inviare — è lo strumento più comodo per costruire il font.



Come si collega tutto al progetto del laboratorio

L'obiettivo (slide 21-22) è trasmettere una lettera alla matrice via SPI usando un **timer interrupt**, alternando tra due lettere (o lettera+simbolo), evitando HAL_Delay. In pratica:

1. Identifica i pin SPI (SCLK, MOSI) e il pin GPIO usato come RCLK del registro (di solito è gestito a mano, non dall'hardware SPI, perché SPI non ha un segnale di "latch" nativo).

2. Configura un timer con interrupt periodico ogni 4 ms. Nella callback: scrivi via SPI i 16 bit (riga+colonna) della colonna corrente e dai un impulso su RCLK; dopo la quinta colonna, ricomincia dalla prima.
3. Usa HAL_SPI_Transmit_DMA per la trasmissione (non bloccante) invece della versione bloccante, così la callback del timer resta breve e non blocca il resto del programma.
4. Serve un meccanismo (es. un contatore/timer separato, o un secondo timer) per decidere quando alternare tra le due lettere da mostrare, mantenendo la routine di interrupt della matrice il più semplice possibile.

Lab09_1

Lab09_1: trasmetti una lettera alla matrice LED usando SPI e una timer interrupt e alterna fra due lettere (o due lettere e un simbolo).

File .ioc

Pin

- SCK: PA5 = SPI1
- MISO: PA6 = SPI1
- MOSI: PA7 = SPI1
- RCLK: PB6 = GPIO_Output

Timer

- Timer 2 (4ms):
  - Clock Source: Internal Clock
  - TIM2 global interrupt: Enabled
  - PSC=99
  - ARR=3359

$$f_{Timer} = \frac{f_{Clock}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(3359+1)(99+1)} = 250\ Hz$$

$$T_{Timer} = \frac{(ARR+1)(PSC+1)}{f_{Clock}} = \frac{(3359+1)(99+1)}{84MHz} = 4ms$$

- Timer 3 (2s):
  - Clock Source: Internal Clock
  - TIM3 global interrupt: Enabled
  - PSC=9999
  - ARR=16799

$$f_{Timer} = \frac{f_{Clock}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(16799+1)(9999+1)} = 0.5\ Hz$$

Connectivity

- SPI1:
  - Mode: Full Duplex Master
  - Data size: 8 bits
  - Prescaler (for Baud Rate): 4
  - DMA:
    - Request: TX (Memory to Peripheral)

- Priority: High
- Mode: Normal

- NVIC:
  - DMA2 stream3 global interrupt = Enabled
  - SPI1 global interrupt = Enabled

Excel

5 × 7 LED matrix design

16	8	4	2	1	
	X	X	X		64
X				X	32
X				X	16
X				X	8
X				X	4
X				X	2
	X	X	X		1

Computed value

COL	16	8	4	2	1
ROW	62	65	65	65	62

Instruction:
Write an 'x' (small letter) where you want the LED to light up.
The corresponding pairs of row and column data appears in the cells in RED (format: decimal)

5 × 7 LED matrix design

16	8	4	2	1	
X				X	64
X			X		32
X		X			16
X	X				8
X		X			4
X			X		2
X				X	1

Computed value

COL	16	8	4	2	1
ROW	127	8	20	34	65

Instruction:
Write an 'x' (small letter) where you want the LED to light up.
The corresponding pairs of row and column data appears in the cells in RED (format: decimal)

Code

Private Define

```
#define NUM_COLS 5
#define NUM_LETTERS 2

#define PORT GPIOB
#define PIN GPIO_PIN_6
```

Private Variables

```
// Definizione Lettere
static const uint8_t LETTER_COL[NUM_LETTERS][NUM_COLS] = {{ 16,  8,  4,  2,  1 },{ 16,
8,  4,  2,  1 }};
```

```
static const uint8_t LETTER_ROW[NUM_LETTERS][NUM_COLS] = {{ 62, 65 , 65 , 65 , 62
},{127 ,8 ,20 ,34 , 65 }}; // "O", "K"

// Contatori
static int col_cnt = 0; // Colonna a cui siamo arrivati = 0:NUM_COLS-1
static int letter_idx=0; //  Lettera (0="O", 1="K") = 0:NUM_LETTERS-1
```

Funzioni (fuori dal Main)

```
static inline void spi_send(uint8_t col, uint8_t row){
	static uint8_t buffer[2]; // 2 byte
	// DAISY RING
	buffer[0]=row; // prima la riga
	buffer[1]=col; // poi la colonna

	// Ogni 4 ms trasmettiamo (refreshamo)
	HAL_SPI_Transmit_DMA(&hspi1, buffer, 2);
}

void HAL_SPI_TxCpltCallback(SPI_HandleTypeDef *hspi){
	if(hspi->Instance==SPI1){
		// Una volta terminata la comunicazione dei 2 byte
		// diamo un impulso su RCLK per far scattare il latch
		// dei due shift register
		HAL_GPIO_WritePin(PORT, PIN, GPIO_PIN_SET);
		HAL_GPIO_WritePin(PORT, PIN, GPIO_PIN_RESET);
	}
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
	// CALLBACK CAMBIO COLONNA (4ms)
    if (htim->Instance == TIM2){
        // refresh colonna (come hai già scritto)
        spi_send(LETTER_COL[letter_idx][col_cnt], LETTER_ROW[letter_idx][col_cnt]);
        col_cnt++;
        if (col_cnt == NUM_COLS){
            col_cnt = 0;
        }
    }
    // CALLBACK CAMBIO LETTERA (2s)
    else if (htim->Instance == TIM3){
        // cambio lettera – vedi punto 4 sotto per una versione più robusta
        letter_idx++;
        if (letter_idx == NUM_LETTERS){
            letter_idx = 0;
        }
    }
}
```

Inizializzazioni (Main, prima del While)

```
  HAL_TIM_Base_Start_IT(&htim2);
  HAL_TIM_Base_Start_IT(&htim3);
```